

Assignment-20.2

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

The screenshot shows a code editor with a Python script named `task1.py` and a sidebar with security analysis results.

Python Script (task1.py):

```
1 import hashlib
2 import json
3 import os
4 import secrets
5 from typing import Dict, Tuple
6
7 USERS_FILE = "users.json"
8 PBKDF2_ITERATIONS = 200_000
9 SALT_BYTES = 16
10
11
12 def load_users() -> Dict[str, Dict[str, str]]:
13     if not os.path.exists(USERS_FILE):
14         return {}
15
16     with open(USERS_FILE, "r", encoding="utf-8") as file:
17         return json.load(file)
18
19
20 def save_users(users: Dict[str, Dict[str, str]]) -> None:
21     with open(USERS_FILE, "w", encoding="utf-8") as file:
22         json.dump(users, file, indent=2)
23
24
25 def hash_password(password: str, salt: bytes) -> str:
26     password_bytes = password.encode("utf-8")
27     hashed = hashlib.pbkdf2_hmac(
28         "sha256",
29         password_bytes,
30         salt,
31         PBKDF2_ITERATIONS,
32         dklen=32,
33     )
34     return hashed.hex()
35
36
37 def create_password_entry(password: str) -> Tuple[str, str]:
38     salt = secrets.token_bytes(SALT_BYTES)
39     salt_hex = salt.hex()
40     hashed_password = hash_password(password, salt)
```

Security Analysis (Python to Java File Handling Conversion Task):

The script is written; now validating syntax since no Python executable is installed in the current Windows environment.

Ran pylanceSyntaxErrors - pylance mcp server (MCP Server)

Considered user security and evaluated password storage options

Secure Password Script Added

- Updated `task1.py`
- The script now:
 - stores user credentials in `users.json`
 - hashes passwords with `hashlib.pbkdf2_hmac("sha256", ...)`
 - uses a unique 16-byte salt per user
 - never stores plain-text passwords

Security risks fixed

- Plain-text password storage allows anyone with file access to read credentials
- If an attacker gains the file, they can reuse passwords on other sites
- No hashing means no protection against offline password cracking
- Using salted PBKDF2 mitigates rainbow-table and brute-force attacks

task1.py

Describe what to build

+ Agent Auto

Local Default Approvals

OUTPUT:

```
KeyboardInterrupt
PS C:\Users\tloke\OneDrive\Desktop\Ai> & "$env:LOCALAPPDATA\Microsoft\WindowsApps\python.exe" -c "import sys; print(sys.version)"
3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
PS C:\Users\tloke\OneDrive\Desktop\Ai> cd c:\Users\tloke\OneDrive\Desktop\Ai ; @"
>> register
>> alice
>> password123
>> login
>> alice
>> password123
>> exit
>> "@ | & "$env:LOCALAPPDATA\Microsoft\WindowsApps\python.exe" task1.py
Simple secure registration and login example.
Enter command (register/login/exit): Username: Password: Registered user: alice
Enter command (register/login/exit): Username: Password: Authentication successful for user: alice
Enter command (register/login/exit): Exiting.
```

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

The screenshot shows a code editor with a Python script on the left and an analysis panel on the right. The script, named `task2.py`, imports `os` and `requests`. It uses `os.environ.get("API_KEY")` to retrieve an API key from environment variables. If the key is not found, it raises a `ValueError`. The script then uses `requests.get` to fetch data from `https://httpbin.org/get` with a Bearer token header. The `main` function prints the URL and the authorization header, and includes an exception handler for `requests.RequestException`.

The right panel, titled "PYTHON TO JAVA FILE HANDLING CONVERSION TASK", shows the results of a security analysis. It states "Task 2 Completed: Secure API Key Management". Under "Initial Script (Hardcoded Key)", it lists two points: the script was created with a hardcoded API key and it used the OpenWeatherMap API. Under "Security Risks Identified", it lists four risks: Credential Exposure, Version Control Risks, Reuse Vulnerabilities, and No Rotation. Under "Refactored Script", it lists four improvements: using environment variables, adding validation, changing the API to httpbin.org, and ensuring the key is never printed or logged.

OUTPUT:

```
Successfully retrieved data from API:
URL: https://httpbin.org/get
Authorization header present: True
```

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated).

- Refactor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

The screenshot shows a code editor with a Python script named `task3.py`. The script implements a basic encryption using `Fernet` and `base64`. The right sidebar contains a document titled "PYTHON TO JAVA FILE HANDLING CONVERSION TASK" with sections for "Initial Weak Encryption Script", "Weaknesses Identified", "Refactored Strong Encryption", and "Validation".

```

1  from cryptography.fernet import Fernet
2  import base64
3
4  def generate_key() -> bytes:
5      # Generate a secure key
6      return Fernet.generate_key()
7
8  def encrypt_data(data: str, key: bytes) -> str:
9      f = Fernet(key)
10     encrypted = f.encrypt(data.encode('utf-8'))
11     return base64.b64encode(encrypted).decode('utf-8')
12
13 def decrypt_data(encrypted_data: str, key: bytes) -> str:
14     f = Fernet(key)
15     encrypted = base64.b64decode(encrypted_data.encode('utf-8'))
16     decrypted = f.decrypt(encrypted).decode('utf-8')
17     return decrypted
18
19 def main():
20     message = input("Enter message to encrypt: ").strip()
21     key = generate_key() # Secure key generation
22
23     encrypted = encrypt_data(message, key)
24     print(f"Encrypted: {encrypted}")
25
26     decrypted = decrypt_data(encrypted, key)
27     print(f"Decrypted: {decrypted}")
28
29     # Demonstrate that without key, it's not easily reversible
30     print("\nAttempting to decrypt without key (will fail):")
31     try:
32         wrong_key = generate_key() # Different key
33         fake_decrypt = decrypt_data(encrypted, wrong_key)
34         print(f"Unexpected success: {fake_decrypt}")
35     except Exception as e:
36         print(f"Failed as expected: {type(e).__name__}")
37
38 if __name__ == "__main__":
39     main()

```

Initial Weak Encryption Script

- Used `base64` encoding as "encryption" - not actual encryption, easily reversible
- Key parameter was ignored entirely
- Anyone could decode the "encrypted" data without any key

Weaknesses Identified

- **No Key Usage:** Encryption didn't depend on the key at all
- **Reversible Encoding:** Base64 is encoding, not encryption - no security
- **No Confidentiality:** Data is readable by anyone with access to the string
- **No Integrity:** No protection against tampering

Refactored Strong Encryption

- Switched to `cryptography.fernet.Fernet` for AES-based symmetric encryption
- Uses secure key generation with `Fernet.generate_key()`
- Proper encryption/decryption with key dependency
- Base64 encoding of encrypted bytes for string representation

Validation

The improved script:

+ task3.py

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 40, Col 1 Spaces: 4 UTF-8 Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

```

Encrypted: gAAAAAB... (long base64 string)
Decrypted: Hello World

Attempting to decrypt without key (will fa
Failed as expected: InvalidToken

```

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

The screenshot displays a code editor with a Python Flask application and a security review panel on the right.

task4.py

```
1 from flask import Flask, request, jsonify
2 import sqlite3
3 import os
4 import hashlib
5 import secrets
6 import re
7
8 app = Flask(__name__)
9
10 # Secure database connection with context manager
11 def get_db():
12     return sqlite3.connect('users.db')
13
14 def hash_password(password: str, salt: bytes) -> str:
15     return hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000).hex()
16
17 def generate_salt() -> bytes:
18     return secrets.token_bytes(16)
19
20 @app.route('/register', methods=['POST'])
21 def register():
22     data = request.get_json()
23     username = data.get('username', '').strip()
24     password = data.get('password', '')
25
26     if not username or not password:
27         return jsonify({'error': 'Username and password required'}), 400
28
29     if len(password) < 8:
30         return jsonify({'error': 'Password must be at least 8 characters'}), 400
31
32     salt = generate_salt()
33     hashed_password = hash_password(password, salt)
34
35     try:
36         with get_db() as conn:
37             cursor = conn.cursor()
38             # Parameterized query prevents SQL injection
39             cursor.execute("INSERT INTO users (username, password_hash, salt) VALUES (?, ?, ?)")
40             cursor.execute("commit")
41     except Exception as e:
42         return jsonify({'error': str(e)}), 500
```

PYTHON TO JAVA FILE HANDLING CONVERSION TASK

Original Insecure Implementation

The initial Flask web service had critical security vulnerabilities in user registration, authentication, file upload, and data retrieval endpoints.

Identified Vulnerabilities

1. **SQL Injection:** String formatting in SQL queries allowed attackers to inject malicious SQL code
2. **Plain Text Password Storage:** Passwords stored and compared without hashing, exposing credentials
3. **Insecure File Upload:** No validation of file types, sizes, or path traversal protection
4. **Information Disclosure:** Passwords exposed in profile endpoint responses
5. **No Input Validation:** Missing checks for malformed or malicious input data

Security Improvements Implemented

- **Parameterized SQL Queries:** Prevents SQL injection using prepared statements
- **Password Hashing:** PBKDF2 with salt for secure password storage and verification
- **File Upload Security:** Type validation, size limits, and path traversal prevention
- **Input Sanitization:** Regex validation and length checks for user inputs
- **Secure Configuration:** Debug mode disabled, proper error handling

task4.py

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 118, Col 1 Spaces: 4 UTF-8 Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

```
Traceback (most recent call last):
  File "task4.py", line 1, in <module>
    from flask import Flask, request, json
ModuleNotFoundError: No module named 'flas
```

To get the actual output:

1. Install Flask: `pip install flask`
2. Run: `python task4.py`

Expected Server Output:

```
* Serving Flask app 'task4'
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Pres
```

Testing the Secured Endpoints:

Once running, you can test with curl:

Register a user:

```
curl -X POST http://127.0.0.1:5000/registre
```

Response: `{"message": "User registered successfully"}`

Login:

```
curl -X POST http://127.0.0.1:5000/login -
```

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

```
task5.py > ...
1 def add_numbers(a, b):
2     return a + b
3
4 def subtract_numbers(a, b):
5     return a - b
6
7 def multiply_numbers(a, b):
8     return a * b
9
10 def divide_numbers(a, b):
11     if b == 0:
12         raise ZeroDivisionError("Cannot divide by zero")
13     return a / b
14
15 def get_number(prompt):
16     while True:
17         try:
18             return float(input(prompt).strip())
19         except ValueError:
20             print("Error: Please enter a valid number.")
21
22 def main():
23     print("Simple Calculator")
24     while True:
25         try:
26             operation = input("Enter operation (+, -, *, /) or 'quit' to exit: ").strip().lower()
27             if operation == 'quit':
28                 print("Goodbye!")
29                 break
30
31             if operation not in ['+', '-', '*', '/']:
32                 print("Error: Invalid operation. Please use +, -, *, or /.")
33                 continue
34
35             num1 = get_number("Enter first number: ")
36             num2 = get_number("Enter second number: ")
37
38             if operation == '+':
39                 result = add_numbers(num1, num2)
```

PYTHON TO JAVA FILE HANDLING CONVERSION TASK

Task 5 Completed: Exception Handling Improvement

Original Program Issues

The initial calculator program crashed on:

- Non-numeric input (ValueError from float())
- Division by zero (ZeroDivisionError)
- Invalid operations (handled poorly)
- No way to exit gracefully

Improved Exception Handling

- **Input Validation:** get_number() function with retry loop for invalid numeric input
- **Operation Validation:** Checks for valid operators with clear error messages
- **Division Protection:** Explicit check for zero divisor
- **General Safety:** Catches unexpected exceptions
- **User Experience:** Loop allows multiple calculations, 'quit' command to exit

Key Improvements

- Try-except blocks around all user input and calculations
- Meaningful error messages instead of crashes
- Input sanitization and validation
- Graceful error recovery without program termination

+ task5.py

Describe what to build

+ Agent Auto

Local Default Approvals

Ln 58, Col 1 Spaces: 4 UTF-8 Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

Simple Calculator

Enter operation (+, -, *, /) or 'quit' to exit: +

Enter first number: 5

Enter second number: 3

Result: 8.0

Enter operation (+, -, *, /) or 'quit' to exit: /

Enter first number: 10

Enter second number: 2

Result: 5.0

Enter operation (+, -, *, /) or 'quit' to exit: /

Enter first number: 10

Enter second number: 0

Error: Cannot divide by zero

Enter operation (+, -, *, /) or 'quit' to exit: invalid

Error: Invalid operation. Please use +, -, *, or /.

Enter operation (+, -, *, /) or 'quit' to exit: quit

Goodbye!